PUBLIC

Code Assessment

of the Stader Withdrawals Smart Contracts

December 11, 2024

Produced for

enzyme

by

CHAINSECURITY

Contents

1 Executive Summary 3
2 Assessment Overview 5
3 Limitations and use of report 7
4 Terminology 8
5 Findings 9
6 Informational 11
7 Notes 12

1 Executive Summary

Dear Enzyme team,

Thank you for trusting us to help Avantgarde Finance with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Stader Withdrawals according to Scope to support you in forming an opinion on their security risks.

Avantgarde Finance implements an Enzyme external positions to handle Stader Withdrawals. Vaults can register an ETHx withdrawal and claim the withdrawn ETH.

The most critical subjects covered in our audit are access control, functional correctness, and the correct integration with the external system. Security regarding integration with the Stader withdrawal manager can be improved, see Inaccurate Pending Withdrawals Valuation. In addition, there is a DoS attack vector regarding the special Stader withdrawal logic, see Register New Withdrawals Can Be DoSed and Withdrawal Queue Can Be Populated to Increase the Gas Cost of getManagedAssets.

In summary, we find that the codebase provides a good level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	3
• Acknowledged	3

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Stader Withdrawals repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	22 Nov 2024	b85907086599c87a75344556165706fcac8a1eed	Initial Version

The following files were in the scope of this review:

```
contracts/
    release/
        extensions/
            external-position-manager/
                external-positions/
                    stader-withdrawals/
                        StaderWithdrawalsPositionLib.sol
                        StaderWithdrawalsPositionParser.sol
```

For the solidity smart contracts, the compiler version `0.8.19` was chosen.

2.1.1 Excluded from scope

All the contracts that are not explicitly mentioned in the Scope section are excluded from scope. Stader is out of scope and expected to work as documented.

2.2 System Overview

This system overview describes the initially received version (Version 1) of the contracts as defined in the Assessment Overview.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Avantgarde Finance offers Stader Withdrawals lib and parser for the Enzyme external positions, which enables redeeming ETHx to Ether.

Stader is a liquid staking protocol, where users stake Ether and receive ETHx as a liquid staking token.

The withdrawals from Stader involve two actions:

1. `requestWithdraw()`: a withdrawal request should be registered first with an ETHx amount to burn and an owner, the beneficiary that receives the Ether eventually. Each withdrawal request is stored in the following struct:

```
struct UserWithdrawInfo {
   address payable owner; // address that can claim eth on behalf of this request
   uint256 ethXAmount; //amount of ethX share locked for withdrawal
   uint256 ethExpected; //eth requested according to given share and exchangeRate
   uint256 ethFinalized; // final eth for claiming according to finalize exchange rate
   uint256 requestBlock; // block number of withdraw request
}
```

2. `claim()`: once finalized, the withdrawal (identifiable by a request id) can be claimed only by the owner, and the finalized Ether will be sent to the owner.

StaderWithdrawalsPositionLib is expected to be used by the ExternalPositionProxy, it implements the following functions to interact with Stader's UserWithdrawalManager for withdrawals:

1. `init()`: This function, without access control in the ExternalPositionProxy, grants `type(uint256).max` ETHx allowance to the UserWithdrawalManager for future withdrawals.
2. `receiveCallFromVault()`: This function is expected to be delegate-called by the ExternalPositionProxy with the VaultProxy as the `msg.sender`. It will request or claim a withdrawal depending on the `actionId`. The ExternalPositionProxy will be the owner of the withdrawal request, which eventually sends the claimed Ether to the VaultProxy.
3. `getDebtAssets()`: Since this external position should not hold any debt, this function returns empty arrays.
4. `getManagedAssets()`: This retrieves the total assets held by the external position that is in the process of Stader withdrawals. It iterates over all the requests of the external position and aggregates the pending ETHx and finalized Ether.

StaderWithdrawalsPositionParser implements the following interfaces:

1. `parseAssetsForAction()`: It returns the assets to transfer and receive depending on the `actionId`. For requesting withdrawals, the asset to transfer will be ETHx, and for claiming requests, the asset to receive will be WETH (by wrapping the received Ether).
2. `parseInitArgs()`: Since there is no storage variable to initialize for the ExternalPositionProxy, this function does nothing.

2.2.1 Roles & Trust Model

Please refer to the main audit report and the extension audit reports for a general trust model of Enzyme protocol and its extensions.

Fund owners and asset managers are generally fully trusted for a fund. However, their powers can be limited through the fund's settings. The funds' settings/policies are assumed to be set up correctly for the intended configuration/usage (e.g. a policy to limit the slippage in 31Third batch trade).

The managers are expected to regularly claim the fees and to pause the position if under-/overvaluation of the fund occurs.

Governance is fully trusted and expected to not only behave honestly but also to fully understand the systems they are interacting which includes choosing appropriate parameters.

Stader is fully trusted and assumed to be honest, otherwise, Enzyme is subject to its risks, for instance:

1. Stader is deployed behind a TransparentUpgradeableProxy. In the worst case, it can be upgraded to a malicious implementation and steal funds.
2. Stader may be configured with bad parameters where withdrawals are restricted.
3. Stader can be paused where no more withdrawal requests can be registered.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Findings

In this section, we describe our findings. The findings are split into these different categories:

- Security: Related to vulnerabilities that could be exploited by malicious actors

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	3

- Inaccurate Pending Withdrawals Valuation Acknowledged
- Register New Withdrawals Can Be DoSed Acknowledged
- Withdrawal Queue Can Be Populated to Increase the Gas Cost of getManagedAssets Acknowledged

5.1 Inaccurate Pending Withdrawals Valuation

Security Low Version 1 Acknowledged

CS-EZSTW-001

In the `getManagedAssets()` function, the unfinalized withdrawals will be valuated according to the requested ETHx amount. In Stader UserWithdrawalManager, the finalized Ether amount for a request is computed based on the current `exchangeRate` and capped by the requiredEth (computed upon request registration):

```
Math.min(requiredEth, (lockedEthX * exchangeRate) / DECIMALS);
```

Theoretically, if there is an abrupt increase of `exchangeRate` after requesting a withdrawal, `getManagedAssets` will overestimate the assets before it is finalized, for instance:

1. requestWithdraw: withdrawn 100 ETHx amount, which implies 100 ETH at the beginning.
2. Now exchange rate increases extremely (1 ETHx = 2 ETH). `getManagedAssets()` will return 100 ETHx worth 200 ETH.
3. When the request is finalized, however, the withdrawal amount is capped by the initial rate, meaning 100 ETH will be withdrawn, and `getManagedAssets()` will return 100 ETH.

Consequently, the valuation returned by `getManagedAssets` may have an abrupt decrease.

Acknowledged:

Avantgarde Finance has acknowledged the caps that is implied on `ethFinalized` per withdrawal request, and monitoring is required on Stader's oracle status and upgrades.

5.2 Register New Withdrawals Can Be DoSed

Security | Low | Version 1 | Acknowledged

CS-EZSTW-002

Stader withdrawal manager is configured with a `maxNonRedeemedUserRequestCount`, which restricts the amount of unredeemed withdrawal requests a user can have. As a result, arbitrary users can push withdrawal requests for the external position, populating the withdrawal requests to the maximal amount. Consequently the external position cannot register its actual withdrawal request for ETHx held by the vault.

Acknowledged:

Avantgarde Finance has acknowledged the potential DoS attack. **Note** this attack vector is due to the design issue of Stader withdrawal manager, hence there is no simple way to patch it in Enzyme external positions.

5.3 Withdrawal Queue Can Be Populated to Increase the Gas Cost of getManagedAssets

Security | Low | Version 1 | Acknowledged

CS-EZSTW-003

Stader withdrawal manager is configured with a `maxNonRedeemedUserRequestCount`, which restricts the amount of unredeemed withdrawal requests a user can have. Arbitrary users can push withdrawal requests for the external position, populating the withdrawal requests to the maximal amount. Consequently, the evaluation of the view function `getManagedAssets` may become expensive as it iterates over all the withdrawal requests for the external position.

Acknowledged:

Avantgarde Finance has acknowledged the potential gas griefing attack. **Note** this attack vector is due to the design issue of Stader withdrawal manager. Stader is assumed to properly set the parameters such as min withdraw amount per request to ensure the attack is economically inviable.

6 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

6.1 A Finalized Request May Have 0 Finalized Ether

Informational **Version 1** **Acknowledged**

CS-EZSTW-004

`getManagedAssets()` distinguishes the status of a withdrawal request by checking if the `ethFinalized` amount is above zero. This is expected to work in most cases, however, theoretically, a finalized request may still have `ethFinalized==0`.

In Stader's UserWithdrawalManager, when a withdrawal request is being processed, the `ethFinalized` will be computed as:

```
Math.min(requiredEth, (lockedEthX * exchangeRate) / DECIMALS);
```

In case the `exchangeRate` drops drastically due to, for instance, heavy slashing on the beacon chain, the `ethFinalized` can be rounded down to 0. Consequently, no assets can be received from this withdrawal request, while `getManagedAssets()` assumes the pending `ethXAmount` is still available. In this case, the falsely assumed pending `ethXAmount` should have a negligible value.

Acknowledged:

Avantgarde Finance has acknowledged the potential case when a finalized withdrawal has 0 `ethFinalized`.

6.2 Gas Optimization

Informational **Version 1** **Acknowledged**

CS-EZSTW-005

In case the external position has no withdraw requests, `getManagedAssets()` creates two new empty arrays to return, though the arrays `assets_` and `amounts_` are already allocated in the return value declaration, which can be returned directly.

```
if (requestIds.length == 0) {
    return (new address[](0), new uint256[](0));
}
```

Acknowledged:

Avantgarde Finance has acknowledged this optimization.

7 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

7.1 Stader Withdrawal Amount Is Restricted

Note **Version 1**

Stader only allows registering withdrawals within a configured range. A withdrawal cannot be registered if the amount is too high or too low. It is worth noting that, if the vault has low amounts of ETHx, no withdrawal will be possible.